

```
from google.colab import files
uploaded = files.upload()
```

No file chosen

Upload widget is only available when the cell has been executed in the current browser session. Please rerun this cell to enable.

```
# Rule-Based Information Extraction – Pure Extraction Version
# Google Colab Ready – Extraction Only (No Metrics)
# For MSDS-692 Capstone: Information Extraction for Event Data
```

```
"""
```

```
Script Name: "rule_based_extraction.py"
Author: Fabienne Van Cappel
Email: fabienne.vancappel@gmail.com
Date: 2025
"""
```

```
import pandas as pd
import re
from datetime import datetime
import warnings
warnings.filterwarnings('ignore')
```

```
# Rule-based information extraction using regex patterns
# Extracts event name, datetime, and venue from text descriptions
class RuleBasedExtractor:
```

```
    # Initialize with regex patterns for each field
    def __init__(self):
```

```
        # Event name extraction patterns (ordered by priority)
```

```
        self.event_patterns = [
```

```
            # Pattern 1: "Event Name at Venue"
```

```
            r'^([^-]+?)\s+at\s+',
```

```
            # Pattern 2: "Event Name – Additional Info"
```

```
            r'^([^-]+?)\s*-\s*',
```

```
            # Pattern 3: Event in quotes
```

```
            r'^"([^-]+)"',
```

```
            # Pattern 4: Title case event before punctuation
```

```
            r'^([A-Z][^!?]*?)(?:\s*[-]\s*)',
```

```
            # Pattern 5: First substantial phrase
```

```
            r'^([^!?\\n]{10,100}?)?:[.!?\\n]|$',
```

```
            # Pattern 6: Everything before common separators
```

```
            r'^([^-|]+?)(?:\s*[-|]\s*)',
```

```
        ]
```

```
    # Venue extraction patterns
```

```
    self.venue_patterns = [
```

```
        # Pattern 1: "at Venue Name"
```

```
        r'\bat\s+([^\n.]+?)(?:\s*[-,]|\s*$)',
```

```
        # Pattern 2: Specific venue types
```

```
        r'\b(?:\w+\s+)*(?:arena|stadium|center|centre|hall|theater|theatre|auditorium|club|bar|speedway|park|amphitheatre|
```

```
        # Pattern 3: Address patterns
```

```
        r'\b(\d+\s+\w+\s+(?:street|st|avenue|ave|road|rd|blvd|boulevard|drive|dr|way|lane|ln))\b',
```

```
        # Pattern 4: "Venue, City" pattern
```

```
        r'\b([A-Z][A-Za-z\s&\\']+?)(?:,\s*(?:Denver|Austin|Daytona|Colorado|Texas|Florida|CO|TX|FL))',
```

```
        # Pattern 5: Location after common prepositions
```

```
        r'(?:(held at|located at|venue:|location:)\s*([^\n.]+?)(?:[,.\n]|$)',
```

```
    ]
```

```
    # DateTime extraction patterns
```

```
    self.datetime_patterns = [
```

```

self.datetime_patterns = [
    # ISO format patterns
    r'\b(\d{4}-\d{2}-\d{2})T\d{2}:\d{2}:\d{2}\b',
    r'\b(\d{4}-\d{2}-\d{2})\b',

    # US date formats
    r'\b(\d{1,2}/\d{1,2}/\d{4})\s*(?:at\s*)?(?:\d{1,2}:\d{2}\s*(?:am|pm|AM|PM))?',
    r'\b(\d{1,2}-\d{1,2}-\d{4})\s*(?:at\s*)?(?:\d{1,2}:\d{2}\s*(?:am|pm|AM|PM))?',

    # Written month formats
    r'\b(?:january|february|march|april|may|june|july|august|september|october|november|december)\s+\d{1,2},?\s+',
    r'\b(?:jan|feb|mar|apr|may|jun|jul|aug|sep|oct|nov|dec)\s+\d{1,2},?\s+\d{4})\s*(?:at\s*)?(?:\d{1,2}:\d{2}\s*(?:am|pm|AM|PM))?',

    # Day of week patterns
    r'\b(?:monday|tuesday|wednesday|thursday|friday|saturday|sunday),?\s*([^\n]+\d{4})\s*(?:at\s*)?(?:\d{1,2}:\d{2}\s*(?:am|pm|AM|PM))?',
    r'\b(?:mon|tue|wed|thu|fri|sat|sun),?\s*([^\n]+\d{4})\s*(?:at\s*)?(?:\d{1,2}:\d{2}\s*(?:am|pm|AM|PM))?',

    # Time-only patterns (when date is elsewhere)
    r'\b(\d{1,2}:\d{2})\s*(?:am|pm|AM|PM)\b',
    r'\b(\d{1,2}:\d{2})\b',
]

# Common false positive filters
self.event_name_stopwords = {
    'eventbrite', 'presents', 'tickets', 'find event', 'view event',
    'save this event', 'share this event', 'follow', 'report', 'contact'
}

self.venue_stopwords = {
    'the', 'and', 'with', 'for', 'presents', 'featuring', 'tickets',
    'find', 'save', 'share', 'follow', 'event', 'details', 'information'
}

# Extract event name using regex patterns
def extract_event_name(self, text):
    if not text or pd.isna(text):
        return ""

    text = str(text).strip()

    # Try each pattern in order of priority
    for i, pattern in enumerate(self.event_patterns):
        match = re.search(pattern, text, re.IGNORECASE | re.MULTILINE)
        if match:
            name = match.group(1).strip()

            # Clean up the extracted name
            name = self._clean_event_name(name)

            # Validate the name
            if self._is_valid_event_name(name):
                return name

    # Fallback: take first reasonable chunk
    fallback = re.split(r'[!?\n]', text)[0]
    fallback = re.split(r'\s*[\—]\s*', fallback)[0]
    fallback = self._clean_event_name(fallback)

    return fallback[:100] if len(fallback) > 5 else ""

# Extract venue using regex patterns
def extract_venue(self, text):
    if not text or pd.isna(text):
        return ""

    text = str(text)

    # Try each venue pattern
    for pattern in self.venue_patterns:
        matches = re.findall(pattern, text, re.IGNORECASE)
        for match in matches:
            if isinstance(match, tuple):
                match = match[0] if match[0] else match[1] if len(match) > 1 else ""

            venue = str(match).strip()
            venue = self._clean_venue_name(venue)

            if self._is_valid_venue(venue):
                return venue

```

```

        return venue

    return ""

# Extract datetime using regex patterns and basic parsing
def extract_datetime(self, text):

    if not text or pd.isna(text):
        return ""

    text = str(text)

    # Try each datetime pattern
    for pattern in self.datetime_patterns:
        matches = re.findall(pattern, text, re.IGNORECASE)
        for match in matches:
            if isinstance(match, tuple):
                # Handle patterns that capture date and time separately
                date_part = match[0] if match[0] else ""
                time_part = match[1] if len(match) > 1 and match[1] else ""
                datetime_str = f"{date_part} {time_part}".strip()
            else:
                datetime_str = str(match)

            # Try to parse and standardize the datetime
            parsed_dt = self._parse_datetime(datetime_str)
            if parsed_dt:
                return parsed_dt

    return ""

# Clean up extracted event name
def _clean_event_name(self, name):

    if not name:
        return ""

    name = str(name).strip()

    # Remove common prefixes/suffixes
    name = re.sub(r'^(eventbrite\s*[\—]\s*)', '', name, flags=re.IGNORECASE)
    name = re.sub(r'\s*[\—]\s*$', '', name)
    name = re.sub(r'\s*presents\s*$', '', name, flags=re.IGNORECASE)

    # Remove extra whitespace
    name = ' '.join(name.split())

    return name

# Clean up extracted venue name
def _clean_venue_name(self, venue):

    if not venue:
        return ""

    venue = str(venue).strip()

    # Remove common noise
    venue = re.sub(r'\s*[\—]\s*$', '', venue)
    venue = re.sub(r'^\s*[\—]\s*', '', venue)

    # Clean up extra whitespace
    venue = ' '.join(venue.split())

    return venue

# Check if extracted event name is valid
def _is_valid_event_name(self, name):

    if not name or len(name) < 3 or len(name) > 200:
        return False

    name_lower = name.lower()

    # Check against stopwords
    if any(stop in name_lower for stop in self.event_name_stopwords):
        return False

```

```

    # Must contain at least one letter
    if not re.search(r'[a-zA-Z]', name):
        return False

    return True

# Check if extracted venue is valid
def _is_valid_venue(self, venue):

    if not venue or len(venue) < 3 or len(venue) > 150:
        return False

    venue_lower = venue.lower()

    # Check against stopwords
    if venue_lower in self.venue_stopwords:
        return False

    # Must contain at least one letter
    if not re.search(r'[a-zA-Z]', venue):
        return False

    return True

# Parse datetime string into ISO format
def _parse_datetime(self, datetime_str):

    if not datetime_str:
        return ""

    datetime_str = datetime_str.strip()

    # Try different parsing approaches
    try:
        # Handle ISO format
        if re.match(r'\d{4}-\d{2}-\d{2}T\d{2}:\d{2}:\d{2}', datetime_str):
            return datetime_str

        if re.match(r'\d{4}-\d{2}-\d{2}', datetime_str):
            return datetime_str + 'T19:00:00'

        # Handle MM/DD/YYYY formats
        if re.match(r'\d{1,2}/\d{1,2}/\d{4}', datetime_str):
            parts = datetime_str.split('/')
            if len(parts) == 3:
                month, day, year = parts
                return f"{year}-{month.zfill(2)}-{day.zfill(2)}T19:00:00"

        # Handle written formats (basic)
        month_map = {
            'jan': '01', 'feb': '02', 'mar': '03', 'apr': '04',
            'may': '05', 'jun': '06', 'jul': '07', 'aug': '08',
            'sep': '09', 'oct': '10', 'nov': '11', 'dec': '12',
            'january': '01', 'february': '02', 'march': '03', 'april': '04',
            'june': '06', 'july': '07', 'august': '08', 'september': '09',
            'october': '10', 'november': '11', 'december': '12'
        }

        for month_name, month_num in month_map.items():
            if month_name in datetime_str.lower():
                # Extract day and year
                day_match = re.search(r'\b(\d{1,2})\b', datetime_str)
                year_match = re.search(r'\b(20\d{2})\b', datetime_str)

                if day_match and year_match:
                    day = day_match.group(1).zfill(2)
                    year = year_match.group(1)
                    return f"{year}-{month_num}-{day}T19:00:00"

    except:
        pass

    return ""

# Extract all three fields from text
def extract_all(self, text):

```

```

    return {
        'event_name': self.extract_event_name(text),
        'datetime': self.extract_datetime(text),
        'venue': self.extract_venue(text)
    }

# Run rule-based extraction on the gold standard dataset
# Returns DataFrame with original data + extracted fields
def run_rule_based_extraction(csv_file_path='gold_standard_200_events.csv', save_results=True):

    try:
        # Load data
        print("Loading gold standard dataset...")
        df = pd.read_csv(csv_file_path)

        print(f"Loaded {len(df)} events")
        print(f"    Cities: {' ', '.join(df['city'].unique())}")
        print(f"    Sources: {' ', '.join(df['data_source'].unique())}")

        # Initialize extractor
        print("\n Initializing rule-based extractor...")
        extractor = RuleBasedExtractor()

        # Storage for predictions
        extracted_names = []
        extracted_datetimes = []
        extracted_venues = []

        print("Running extraction...")

        # Process each event
        for idx, row in df.iterrows():
            text = row['raw_description']

            # Extract all fields
            result = extractor.extract_all(text)

            extracted_names.append(result['event_name'])
            extracted_datetimes.append(result['datetime'])
            extracted_venues.append(result['venue'])

            # Show progress
            if (idx + 1) % 50 == 0:
                print(f"    Processed {idx + 1}/{len(df)} events...")

        print(f"Extraction complete! Processed {len(df)} events")

        # Add extracted fields to dataframe
        df['rule_based_event_name'] = extracted_names
        df['rule_based_datetime'] = extracted_datetimes
        df['rule_based_venue'] = extracted_venues

        # Save results if requested
        if save_results:
            output_file = 'rule_based_extraction_results.csv'
            df.to_csv(output_file, index=False)
            print(f"Results saved to: {output_file}")

        return df

    except FileNotFoundError:
        print(f"Error: Could not find file '{csv_file_path}'")
        print("    Make sure to upload your gold standard CSV to Colab first!")
        return None
    except Exception as e:
        print(f"Error during extraction: {e}")
        return None

# Show sample extractions for manual review
def show_sample_extractions(df, n_examples=10):

    print(f"\n Sample Rule-Based Extractions (first {n_examples} events):")
    print("=" * 100)

    for i in range(min(n_examples, len(df))):
        row = df.iloc[i]

```

```

print(f"\nEvent {i+1} ({row['city']}):")
print(f"Raw Text: {row['raw_description'][:120]}...")
print(f"└─ Event Name:")
print(f"    └─ Extracted: '{row['rule_based_event_name']}'")
print(f"    └─ Verified: '{row['verified_event_name']}'")
print(f"└─ DateTime:")
print(f"    └─ Extracted: '{row['rule_based_datetime']}'")
print(f"    └─ Verified: '{row['verified_start_datetime']}'")
print(f"└─ Venue:")
print(f"    └─ Extracted: '{row['rule_based_venue']}'")
print(f"    └─ Verified: '{row['verified_venue']}'")

# Print summary statistics of extraction results
def extraction_summary(df):

    print(f"\n Rule-Based Extraction Summary:")
    print("=" * 50)

    total_events = len(df)

    # Count non-empty extractions
    extracted_names = sum(1 for x in df['rule_based_event_name'] if x and str(x).strip())
    extracted_datetimes = sum(1 for x in df['rule_based_datetime'] if x and str(x).strip())
    extracted_venues = sum(1 for x in df['rule_based_venue'] if x and str(x).strip())

    print(f"Total Events: {total_events}")
    print(f"Event Names Extracted: {extracted_names} ({extracted_names/total_events:.1%})")
    print(f"DateTimes Extracted: {extracted_datetimes} ({extracted_datetimes/total_events:.1%})")
    print(f"Venues Extracted: {extracted_venues} ({extracted_venues/total_events:.1%})")

    # Count by difficulty
    if 'annotation_difficulty' in df.columns:
        print(f"\nExtraction by Difficulty:")
        for difficulty in ['Easy', 'Medium', 'Hard']:
            subset = df[df['annotation_difficulty'] == difficulty]
            if len(subset) > 0:
                names_pct = sum(1 for x in subset['rule_based_event_name'] if x and str(x).strip()) / len(subset)
                print(f"    {difficulty}: {names_pct:.1%} event names extracted")

# Main execution (in Google Colab)
# Main function to run rule-based extraction
# Just extraction (no metrics)
def main():

    # Run extraction
    df = run_rule_based_extraction()

    if df is not None:
        # Show summary
        extraction_summary(df)

        # Show sample results
        show_sample_extractions(df, n_examples=5)

        print(f"\n Rule-based extraction complete!")
        print(f"Results saved in DataFrame with new columns:")
        print(f"    - rule_based_event_name")
        print(f"    - rule_based_datetime")
        print(f"    - rule_based_venue")

    return df

return None

```

```
df_with_extractions = main()
```

```

└ Venue:
  └ Extracted: 'Click on the image to go to Q2 Stadium'
  └ Verified: ' Q2 Stadium in Austin, Texas, Soccer'

```

Event 2 (Austin):

Raw Text: Action Navigation

Saturday, May 17, 2025 – 7:30 PM to 11:59 PM (Click on the image to go to Q2 Stadium's mobility infor...

```

└ Event Name:
  └ Extracted: 'Action Navigation Saturday, May 17, 2025'
  └ Verified: 'Major League Soccer – Austin vs. Kansas City'
└ DateTime:
  └ Extracted: '2025-05-17T19:00:00'
  └ Verified: '2024-05-17T18:30-23:59'
└ Venue:
  └ Extracted: 'Click on the image to go to Q2 Stadium'
  └ Verified: ' Q2 Stadium in Austin, Texas, Soccer'

```

Event 3 (Austin):

Raw Text: Action Navigation

Wednesday, October 1, 2025 – Friday, October 31, 2025 – 6:00 PM to 12:00 AM Ana Gabriel Friday / Oct ...

```

└ Event Name:
  └ Extracted: 'Action Navigation Wednesday, October 1, 2025'
  └ Verified: 'Ana Gabriel'
└ DateTime:
  └ Extracted: '2025-10-01T19:00:00'
  └ Verified: '2025-10-01T18:00-11:59'
└ Venue:
  └ Extracted: 'Moody Center Accessibility Guide Location Information Moody Center'
  └ Verified: 'Claro de Luna'

```

Event 4 (Austin):

Raw Text: Austin Country Music Festival at Leander– Cedar Park, April 6, 2024 – Saturday, April 6, 2024 at The Fieldhouse

```

└ Event Name:
  └ Extracted: 'Austin Country Music Festival'
  └ Verified: 'Austin Country Music Festival '
└ DateTime:
  └ Extracted: '2024-04-06T19:00:00'
  └ Verified: '2024-04-06T12:00-23:59'
└ Venue:
  └ Extracted: 'Leander'
  └ Verified: 'The Fieldhouse Amphitheater '

```

Event 5 (Austin):

Raw Text: Austin Music Love Fest – Free Songwriter Festival – Saturday, March 23, 2024 at Thicket Food Park, Austin, TX.

```

└ Event Name:

```

```

"""

```

Information Extraction Metrics Calculator
Calculates Precision, Recall, and F1 scores for rule-based extraction results
Compares extracted fields against manually verified ground truth

```

"""

```

```

import pandas as pd
import numpy as np
import re
from typing import Dict, List, Tuple

```

```

# Calculate precision, recall, and F1 for information extraction tasks
class ExtractionMetrics:

```

```

    # Initialize with DataFrame containing both extracted and verified fields
    def __init__(self, df):
        self.df = df
        self.results = {}

```

```

    # Normalize text for comparison
    def normalize_text(self, text):

```

```

        if pd.isna(text) or text == "":
            return ""

```

```

        text = str(text).lower().strip()

```

```

        # Remove extra whitespace
        text = ' '.join(text.split())

```

```

        # Remove common punctuation
        text = re.sub(r'[^\w\s]', ' ', text)
        text = ' '.join(text.split())

```

```

        return text

# After normalization
# Check if predicted exactly matches actual
def exact_match(self, predicted, actual):
    pred_norm = self.normalize_text(predicted)
    actual_norm = self.normalize_text(actual)

    if not pred_norm and not actual_norm:
        return True # Both empty counts as match

    return pred_norm == actual_norm

# Check if predicted partially matches actual
def partial_match(self, predicted, actual):

    pred_norm = self.normalize_text(predicted)
    actual_norm = self.normalize_text(actual)

    if not pred_norm or not actual_norm:
        return pred_norm == actual_norm # Only match if both empty

    # Check if one contains the other
    return pred_norm in actual_norm or actual_norm in pred_norm

# Calculate precision, recall, F1 for a specific field
def calculate_field_metrics(self, predicted_col, actual_col, match_type='exact'):

    predictions = self.df[predicted_col].tolist()
    actuals = self.df[actual_col].tolist()

    if len(predictions) != len(actuals):
        raise ValueError("Predictions and actuals must have same length")

    # Count matches and totals
    matches = 0
    predicted_non_empty = 0
    actual_non_empty = 0

    for pred, actual in zip(predictions, actuals):

        # Count non-empty predictions and actuals
        if self.normalize_text(pred):
            predicted_non_empty += 1
        if self.normalize_text(actual):
            actual_non_empty += 1

        # Check for match
        if match_type == 'exact':
            if self.exact_match(pred, actual):
                matches += 1
        elif match_type == 'partial':
            if self.partial_match(pred, actual):
                matches += 1

    # Calculate metrics
    precision = matches / predicted_non_empty if predicted_non_empty > 0 else 0
    recall = matches / actual_non_empty if actual_non_empty > 0 else 0
    f1 = 2 * (precision * recall) / (precision + recall) if (precision + recall) > 0 else 0

    return {
        'precision': precision,
        'recall': recall,
        'f1': f1,
        'matches': matches,
        'predicted_non_empty': predicted_non_empty,
        'actual_non_empty': actual_non_empty,
        'total_samples': len(predictions)
    }

# Special handling for datetime metrics - extracts dates for comparison
def calculate_datetime_metrics(self, predicted_col, actual_col):

    predictions = self.df[predicted_col].tolist()
    actuals = self.df[actual_col].tolist()

    # Extract iust the date part for comparison

```



```

def extract_date(datetime_str):

    if not datetime_str or pd.isna(datetime_str):
        return ""

    datetime_str = str(datetime_str)

    # Search for date patterns
    date_match = re.search(r'(\d{4}-\d{2}-\d{2})', datetime_str)
    if date_match:
        return date_match.group(1)

    # Try other common formats
    date_match = re.search(r'(\d{1,2}/\d{1,2}/\d{4})', datetime_str)
    if date_match:

        # Convert to YYYY-MM-DD format
        month, day, year = date_match.group(1).split('/')
        return f"{year}-{month.zfill(2)}-{day.zfill(2)}"

    return ""

# Extract dates for comparison
pred_dates = [extract_date(pred) for pred in predictions]
actual_dates = [extract_date(actual) for actual in actuals]

# Calculate metrics on dates
matches = 0
predicted_non_empty = 0
actual_non_empty = 0

for pred_date, actual_date in zip(pred_dates, actual_dates):
    if pred_date:
        predicted_non_empty += 1
    if actual_date:
        actual_non_empty += 1

    if pred_date and actual_date and pred_date == actual_date:
        matches += 1

precision = matches / predicted_non_empty if predicted_non_empty > 0 else 0
recall = matches / actual_non_empty if actual_non_empty > 0 else 0
f1 = 2 * (precision * recall) / (precision + recall) if (precision + recall) > 0 else 0

return {
    'precision': precision,
    'recall': recall,
    'f1': f1,
    'matches': matches,
    'predicted_non_empty': predicted_non_empty,
    'actual_non_empty': actual_non_empty,
    'total_samples': len(predictions)
}

# Calculate metrics for all fields: event_names, datetime, venue
def calculate_all_metrics(self):

    print("Calculating Information Extraction Metrics...")
    print("=" * 60)

    # Define field mappings
    fields = {
        'event_name': {
            'predicted': 'rule_based_event_name',
            'actual': 'verified_event_name'
        },
        'datetime': {
            'predicted': 'rule_based_datetime',
            'actual': 'verified_start_datetime'
        },
        'venue': {
            'predicted': 'rule_based_venue',
            'actual': 'verified_venue'
        }
    }

    results = {}

```

```

for field_name, columns in fields.items():
    print(f"Processing {field_name}...")

    if field_name == 'datetime':

        # Special handling for datetime
        exact_metrics = self.calculate_datetime_metrics(
            columns['predicted'],
            columns['actual']
        )

        # For datetime, partial match is exact for dates
        partial_metrics = exact_metrics.copy()
    else:

        # Regular text processing field
        exact_metrics = self.calculate_field_metrics(
            columns['predicted'],
            columns['actual'],
            match_type='exact'
        )
        partial_metrics = self.calculate_field_metrics(
            columns['predicted'],
            columns['actual'],
            match_type='partial'
        )

    results[field_name] = {
        'exact': exact_metrics,
        'partial': partial_metrics
    }

self.results = results
return results

# Print formatted results in a table (i.e. tabulation)
def print_results_table(self):

    print("\n" + "=" * 80)
    print("RULE-BASED INFORMATION EXTRACTION METRICS")
    print("=" * 80)

    # Headers
    print(f"{'Field':<12} {'Match Type':<10} {'Precision':<10} {'Recall':<10} {'F1':<10} {'Matches':<8} {'Pred/Actual'}")
    print("-" * 80)

    for field_name, metrics in self.results.items():
        for match_type in ['exact', 'partial']:
            m = metrics[match_type]
            pred_actual = f"{m['predicted_non_empty']}/{m['actual_non_empty']}"

            print(f"{'field_name':<12} {'match_type':<10} {'m['precision']':<10.3f} {'m['recall']':<10.3f} "
                  f"{'m['f1']':<10.3f} {'m['matches']':<8} {'pred_actual':<12}")

# Print summary statistics
def print_summary_statistics(self):

    print(f"\nSUMMARY STATISTICS:")
    print("-" * 40)

    # Averages calculation
    exact_f1_scores = [self.results[field]['exact']['f1'] for field in self.results]
    partial_f1_scores = [self.results[field]['partial']['f1'] for field in self.results]

    print(f"Average Exact F1: {np.mean(exact_f1_scores):.3f}")
    print(f"Average Partial F1: {np.mean(partial_f1_scores):.3f}")

    # What are the Best and worst performing fields?
    best_field_exact = max(self.results.keys(), key=lambda x: self.results[x]['exact']['f1'])
    worst_field_exact = min(self.results.keys(), key=lambda x: self.results[x]['exact']['f1'])

    print(f"\nBest Field (Exact F1): {best_field_exact} ({self.results[best_field_exact]['exact']['f1']:.3f})")
    print(f"Worst Field (Exact F1): {worst_field_exact} ({self.results[worst_field_exact]['exact']['f1']:.3f})")

    # Statistics
    print(f"\nCOVERAGE STATISTICS:")
    print("-" * 40)

```

```

total_events = len(self.df)

for field_name in self.results:
    predicted_coverage = self.results[field_name]['exact']['predicted_non_empty'] / total_events
    actual_coverage = self.results[field_name]['exact']['actual_non_empty'] / total_events

    print(f"{field_name:<12}: {predicted_coverage:.1%} extracted, {actual_coverage:.1%} have ground truth")

# Performance analysis for annotation
def analyze_errors_by_difficulty(self):

    if 'annotation_difficulty' not in self.df.columns:
        print("\nAnnotation difficulty data not available for error analysis.")
        return

    print(f"\nPERFORMANCE BY DIFFICULTY:")
    print("-" * 40)

    difficulties = ['Easy', 'Medium', 'Hard']

    for difficulty in difficulties:
        subset = self.df[self.df['annotation_difficulty'] == difficulty]
        if len(subset) == 0:
            continue

        print(f"\n{difficulty} Events ({len(subset)} events):")

        # Calculate metrics for this subset
        subset_metrics = ExtractionMetrics(subset)
        subset_results = subset_metrics.calculate_all_metrics()

        for field_name in subset_results:
            f1 = subset_results[field_name]['exact']['f1']
            matches = subset_results[field_name]['exact']['matches']
            total = len(subset)
            print(f"  {field_name:<12}: F1={f1:.3f} ({matches}/{total} exact matches)")

# Show examples of extraction errors for analysis
def show_error_examples(self, field_name, n_examples=5):

    print(f"\nERROR EXAMPLES - {field_name.upper()}:")
    print("-" * 60)

    field_mapping = {
        'event_name': ('rule_based_event_name', 'verified_event_name'),
        'datetime': ('rule_based_datetime', 'verified_start_datetime'),
        'venue': ('rule_based_venue', 'verified_venue')
    }

    if field_name not in field_mapping:
        print(f"Invalid field name: {field_name}")
        return

    pred_col, actual_col = field_mapping[field_name]

    # Find errors (non-exact matches)
    errors = []
    for idx, row in self.df.iterrows():
        if not self.exact_match(row[pred_col], row[actual_col]):
            errors.append(idx)

    print(f"Found {len(errors)} errors out of {len(self.df)} total examples")

    # Show first n examples
    for i, idx in enumerate(errors[:n_examples]):
        row = self.df.iloc[idx]
        print(f"\nError {i+1} (Row {idx}):")
        print(f"Raw Text: {row['raw_description'][:100]}...")
        print(f"Extracted: '{row[pred_col]}'")
        print(f"Verified: '{row[actual_col]}'")

# Main function for calculation of metrics
def run_metrics_calculation(csv_file='rule_based_extraction_results.csv'):

    try:
        # Load the extraction results
        print("Loading extraction results...")

```

```

df = pd.read_csv(csv_file)

print(f"Loaded {len(df)} events with extraction results")

# Check required columns
required_cols = [
    'rule_based_event_name', 'rule_based_datetime', 'rule_based_venue',
    'verified_event_name', 'verified_start_datetime', 'verified_venue'
]

missing_cols = [col for col in required_cols if col not in df.columns]
if missing_cols:
    print(f"Error: Missing required columns: {missing_cols}")
    return None

# Calculate metrics
metrics_calc = ExtractionMetrics(df)
results = metrics_calc.calculate_all_metrics()

# Display results
metrics_calc.print_results_table()
metrics_calc.print_summary_statistics()
metrics_calc.analyze_errors_by_difficulty()

print(f"\nMETRICS CALCULATION COMPLETE!")
print(f"Results show rule-based extraction performance across all fields.")

return metrics_calc, results

except FileNotFoundError:
    print(f"Error: Could not find file '{csv_file}'")
    print("Make sure you've run the rule-based extraction first!")
    return None
except Exception as e:
    print(f"Error during metrics calculation: {e}")
    return None

# Helper function: show error examples for a specific field
def show_field_errors(metrics_calc, field_name='event_name', n_examples=5):

    if metrics_calc:
        metrics_calc.show_error_examples(field_name, n_examples)

# Main execution (in Google Colab)
# The complete metrics calculation
def main():

    metrics_calc, results = run_metrics_calculation()

    if metrics_calc:
        print(f"\nTo see error examples for specific fields, use:")
        print(f"show_field_errors(metrics_calc, 'event_name', 5)")
        print(f"show_field_errors(metrics_calc, 'datetime', 5)")
        print(f"show_field_errors(metrics_calc, 'venue', 5)")

    return metrics_calc, results

# For Google Colab, run:
# metrics_calc, results = main()

# Set env in working dir and from home directory scripts
# python3 scripts/metrics_calculator.py

```

```
metrics_calc, results = main()
```

```
-----
Field          Match Type Precision Recall    F1          Matches  Pred/Actual

```

```
Average Exact F1:    0.278
```

Average Partial F1: 0.399

Best Field (Exact F1): datetime (0.550)
Worst Field (Exact F1): event_name (0.140)

COVERAGE STATISTICS:

```
event_name : 100.0% extracted, 100.0% have ground truth
datetime   : 64.0% extracted, 99.5% have ground truth
venue      : 89.0% extracted, 100.0% have ground truth
```

PERFORMANCE BY DIFFICULTY:

Easy Events (71 events):
Calculating Information Extraction Metrics...

```
=====
Processing event_name...
Processing datetime...
Processing venue...
event_name : F1=0.239 (17/71 exact matches)
datetime   : F1=0.742 (49/71 exact matches)
venue      : F1=0.219 (15/71 exact matches)
```

Medium Events (67 events):
Calculating Information Extraction Metrics...

```
=====
Processing event_name...
Processing datetime...
Processing venue...
event_name : F1=0.060 (4/67 exact matches)
datetime   : F1=0.467 (25/67 exact matches)
venue      : F1=0.144 (9/67 exact matches)
```

Hard Events (50 events):
Calculating Information Extraction Metrics...

```
=====
Processing event_name...
Processing datetime...
Processing venue...
event_name : F1=0.140 (7/50 exact matches)
datetime   : F1=0.188 (6/50 exact matches)
venue      : F1=0.065 (3/50 exact matches)
```

METRICS CALCULATION COMPLETE!
Results show rule-based extraction performance across all fields.

show_field_errors(metrics_calc, 'event_name', 10)

ERROR EXAMPLES – EVENT_NAME:

Found 172 errors out of 200 total examples

Error 1 (Row 0):
Raw Text: Action Navigation

Saturday, May 18, 2024 – 7:30 PM to 11:59 PM (Click on the image to go to Q2 Stad...
Extracted: 'Action Navigation Saturday, May 18, 2024'
Verified: 'Major League Soccer – Austin vs. Kansas City'

Error 2 (Row 1):
Raw Text: Action Navigation

Saturday, May 17, 2025 – 7:30 PM to 11:59 PM (Click on the image to go to Q2 Stad...
Extracted: 'Action Navigation Saturday, May 17, 2025'
Verified: 'Major League Soccer – Austin vs. Kansas City'

Error 3 (Row 2):
Raw Text: Action Navigation

Wednesday, October 1, 2025 – Friday, October 31, 2025 – 6:00 PM to 12:00 AM Ana G...
Extracted: 'Action Navigation Wednesday, October 1, 2025'
Verified: 'Ana Gabriel'

Error 4 (Row 4):
Raw Text: Austin Music Love Fest – Free Songwriter Festival – Saturday, March 23, 2024 at Thicket Food Park, A...
Extracted: 'Austin Music Love Fest'
Verified: 'South Austin Music Festival '

Error 5 (Row 7):
Raw Text: Austin FC Watch Party at MVP SoCo – Saturday, March 23, 2024 at My Vinyasa Practice SoCo, Austin, TX...
Extracted: 'Austin FC Watch Party'

Verified: 'Soccer Austin FC vs. Orlando'

Error 6 (Row 8):

Raw Text: Action Navigation

Saturday, July 6, 2024 – 7:30 PM to 11:59 PM (Click on the image to go to Q2 Stad...

Extracted: 'Action Navigation Saturday, July 6, 2024'

Verified: 'Soccer Austin FC vs. New York City FC'

Error 7 (Row 9):

Raw Text: Action Navigation

Saturday, October 19, 2024 – 8:00 PM to 11:59 PM (Click on the image to go to Q2 ...

Extracted: 'Action Navigation Saturday, October 19, 2024'

Verified: 'Soccer Austin FC vs. Colorado Rapids'

Error 8 (Row 10):

Raw Text: Action Navigation

Sunday, March 23, 2025 – 3:00 PM to 11:59 PM (Click on the image to go to Q2 Stad...

Extracted: 'Action Navigation Sunday, March 23, 2025'

Verified: 'Soccer Austin FC vs. San Diego FC'

Error 9 (Row 11):

show_field_errors(metrics_calc, 'venue', 10)

Error 1 (Row 0):

Raw Text: Action Navigation

Saturday, May 18, 2024 – 7:30 PM to 11:59 PM (Click on the image to go to Q2 Stad...

Extracted: 'Click on the image to go to Q2 Stadium'

Verified: 'Q2 Stadium in Austin, Texas, Soccer'

Error 2 (Row 1):

Raw Text: Action Navigation

Saturday, May 17, 2025 – 7:30 PM to 11:59 PM (Click on the image to go to Q2 Stad...

Extracted: 'Click on the image to go to Q2 Stadium'

Verified: 'Q2 Stadium in Austin, Texas, Soccer'

Error 3 (Row 2):

Raw Text: Action Navigation

Wednesday, October 1, 2025 – Friday, October 31, 2025 – 6:00 PM to 12:00 AM Ana G...

Extracted: 'Moody Center Accessibility Guide Location Information Moody Center'

Verified: 'Claro de Luna'

Error 4 (Row 3):

Raw Text: Austin Country Music Festival at Leander– Cedar Park, April 6, 2024 – Saturday, April 6, 2024 at The...

Extracted: 'Leander'

Verified: 'The Fieldhouse Amphitheater'

Error 5 (Row 4):

Raw Text: Austin Music Love Fest – Free Songwriter Festival – Saturday, March 23, 2024 at Thicket Food Park, A...

Extracted: 'Thicket Food Park'

Verified: 'Austin's Moontower Saloon'

Error 6 (Row 5):

Raw Text: Austin Country Music Festival at Leander– Cedar Park, April 6, 2024 – Saturday, April 6, 2024 at The...

Extracted: 'Leander'

Verified: 'Leander – Cedar Park, Austin'

Error 7 (Row 6):

Raw Text: Austin Music Love Fest – Free Songwriter Festival – Saturday, March 23, 2024 at Thicket Food Park, A...

Extracted: 'Thicket Food Park'

Verified: 'Thicket Food Park, Austin'

Error 8 (Row 7):

Raw Text: Austin FC Watch Party at MVP SoCo – Saturday, March 23, 2024 at My Vinyasa Practice SoCo, Austin, TX...

Extracted: 'MVP SoCo'

Verified: 'MVP SoCo (My Vinyasa Practice SoCo), Austin, TX'

Error 9 (Row 8):

Raw Text: Action Navigation

Saturday, July 6, 2024 – 7:30 PM to 11:59 PM (Click on the image to go to Q2 Stad...

Extracted: 'Click on the image to go to Q2 Stadium'

Saturday, October 19, 2024 – 8:00 PM to 11:59 PM (Click on the image to go to Q2 ...
Extracted: 'Click on the image to go to 02 Stadium'

Start coding or [generate](#) with AI.